

Below Zero

One Step At a Time

In this documentation, we will learn about **Deep Neural Nets** and how they work under the hood. We are going to create a **Neural Network** from scratch without using any library, and we will use it to predict the famous **MNIST** *handwritten digits classification* dataset.

*This documentation is recommended for those people who already have prior knowledge of **Deep Neural Networks** and have a bit of experience in industry-standard libraries like **TensorFlow** or **PyTorch** also they are comfortable working with **C**, specifically **structures**, **pointers**, and **memory allocations**.*

What is a Neural Network?

A neural network is a machine learning model to process data, recognize hidden patterns, and make predictions. More technically, it is a collection of mathematical expressions.

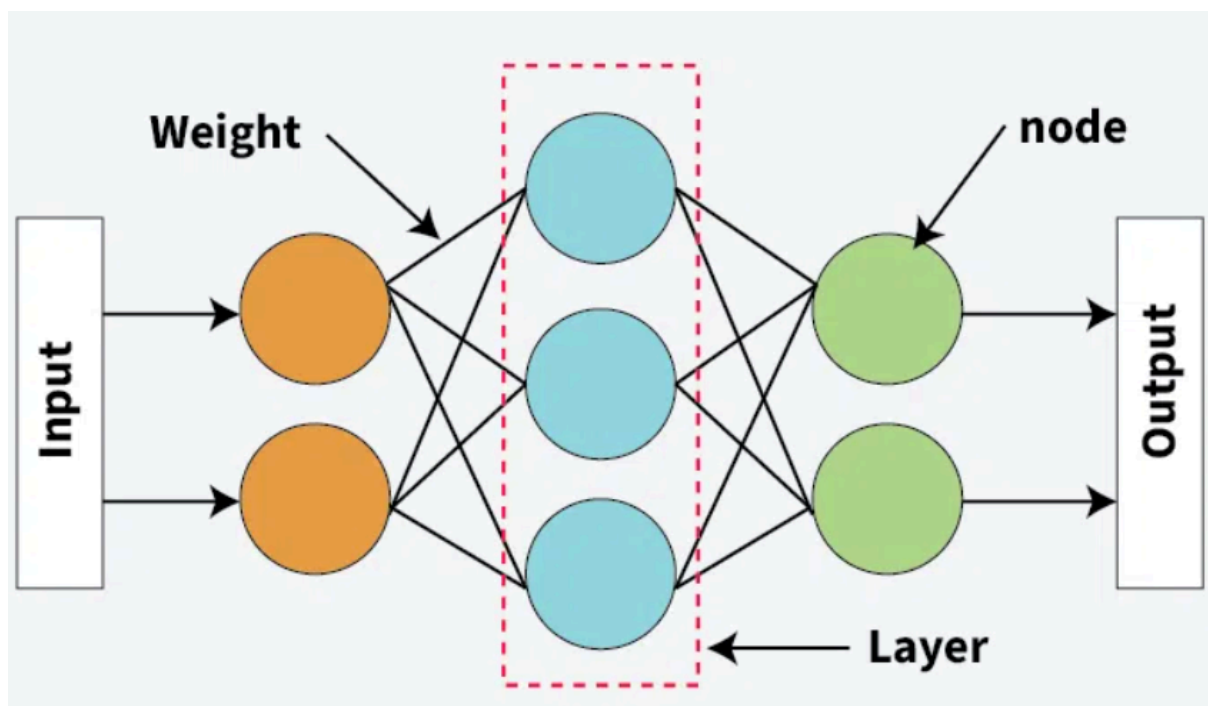


Fig 1.0 ~ shows an image of a neural network

How does it work?

A neural network learns by passing data through a continuous cycle of mathematical operations, measuring its mistakes, and adjusting its internal parameters to do better next time. When stripping away the abstractions of modern libraries, the mechanics of a neural network boil down to four primary phases:

Forward Propagation:

Data enters the input layer and flows through the hidden layers to produce an output prediction. Computationally, this is a series of matrix multiplications and additions:

$$Z = W \cdot X + b$$

(where W is the weight matrix, X is the input matrix, and b is the bias vector)

The result is then passed through an activation function, such as **ReLU** or **Sigmoid**, to introduce non-linearity, allowing the network to map complex, non-linear relationships.

Loss Calculation:

Once the network outputs a prediction, we compare it to the original target data or the actual output. A Loss Function (such as Categorical Cross-Entropy for classification tasks) computes a single scalar value that quantifies how far the prediction is from the target. The entire objective of the network is to minimize this specific number.

Backpropagation:

This is a mathematical engine that uses the chain rule of calculus. The network works backward from the output layer to the input layer, calculating the gradient (the partial derivative) of the loss with respect to every single weight and bias. In this scratch implementation, we manually derive and implement the equations to figure out exactly how much each parameter contributed to the final error.

Gradient Descent:

Finally, the network updates its parameters to reduce the error for the next forward pass. Using an optimization algorithm, we adjust the weights and biases by subtracting a small fraction (determined by the learning rate, α) of their respective gradients:

$$W = W - \alpha \cdot \partial W$$

By repeating this four-step cycle over thousands of iterations, the network's weights and biases gradually converge to an optimal state, transforming it from a random number generator into a highly accurate digit classifier.

Building from the Ground Up

To build a neural network in pure C, we need to define our data structures. We will use a strictly hierarchical approach, moving from the smallest unit to the complete network. We need three core blueprints:

- **Neuron:** The fundamental processing unit. It holds its own weights, bias, and output values.
- **Layer:** A collection (array) of *Neurons*

- **MLP (Multi-Layer Perceptron):** A collection (array) of *Layers* that form the complete network.

Before we can build the MLP, the most sensible place to start is understanding how a single Neuron calculates its output.

The Math of a Single Neuron:

A single neuron performs a very straightforward mathematical operation. It takes multiple inputs, scales each input by a specific weight, sums them, adds a bias, and passes the result through an activation function.

Mathematically, for a neuron with n inputs, the raw output z is calculated as the dot product of the inputs and weights, plus the bias:

$$Z = \sum_{i=1}^n (x_i \cdot w_i) + b$$

Once we have Z , we pass it through an activation function (like ReLU) to get the final activated output :

$$a = \sigma(Z)$$

(Where σ is an activation function)

- **X** = The input values (either from the dataset or the previous layer).
- **W** = The weights (how much importance this neuron gives to each specific input).
- **b** = The bias (a baseline adjustment to shift the activation threshold).

In our code, we will use the **hyperbolic tangent** for the activation function. So let's learn about it

Hyperbolic Tangent (tanh):

It is a [hyperbolic function](#) that is the analogue of the [tangent](#) function for [hyperbolic spaces](#), taking in a [hyperbolic angle](#) as an [argument](#) and returning the y -coordinate for the corresponding point on the [unit hyperbola](#), symbolically represented as:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

It is an S-shaped (sigmoid) curve that takes any real-valued number and "squashes" it into a range between **-1.0 and 1.0**. Because its outputs range from negative to positive, Tanh is naturally zero-centered. This makes it an excellent choice for hidden layers, as it helps the optimization algorithm converge faster.

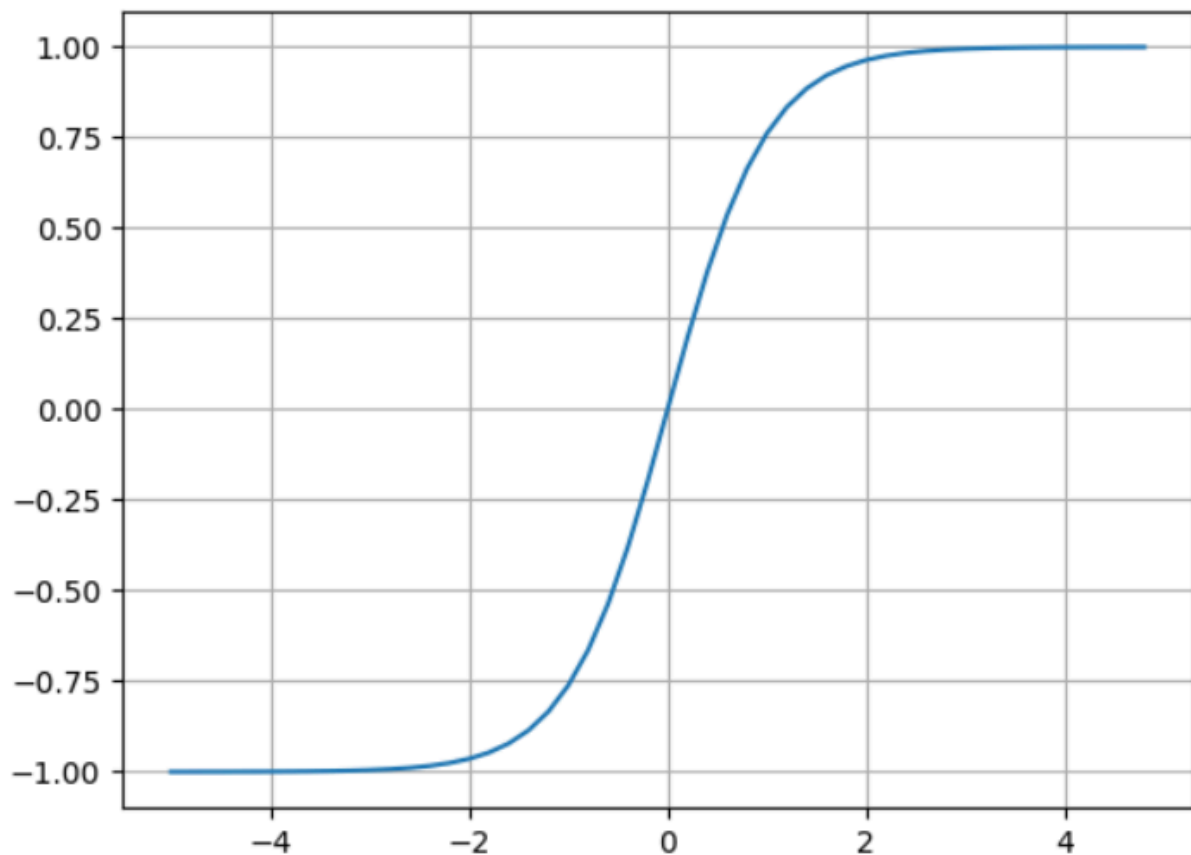


Fig 1.1 ~ shows the curve of the hyperbolic tangent

Now it's time to implement this on a single neuron in **C**.

Step-by-step Implementation:

First, we need a **structure** for our single neuron, which will contain weights and bias.

```
// Definition of our Neuron structure
typedef struct {
    int input_dim;
    double *w;
    double b;
}Neuron;
```

Here, input_dim will be our input dimensions, and *w will be the input tensor, and b will be the bias.

```
// Initializes the neuron with random weights and bias
void init_neuron(Neuron *n, int input_dim){
    n->input_dim = input_dim;
    n->w = (double*) malloc(input_dim * sizeof(double));
```

```

    for(int i = 0; i < input_dim; i++){
        n->w[i] = ((double)rand() / RAND_MAX) * 2.0 - 1.0; // Random weights
        // between -1 and 1
    }
    n->b = ((double)rand() / RAND_MAX) * 2.0 - 1.0; // Random bias between
    // -1 and 1
}

```

Clearly, we created an *init function* and initialized our Neuron with random weights and bias. In neural networks, we want weights to be both positive *and* negative (typically centered around zero). A better C trick for random floats between -1.0 and 1.0 is:

```
((double)rand() / RAND_MAX) * 2.0 - 1.0
```

Now we will begin the math,

```

// Calculates the output by our formula: output = w1*x1 + w2*x2 + ... +
wn*xn + b
double call_neuron(Neuron *n, double *in){
    double sum = 0;
    for(int i = 0; i < n->input_dim; i++){
        sum += n->w[i] * in[i];
    }
    return sum + n->b;
}

```

Here, we are using the fundamental formula to return an output. Now, we need to pass that output to an activation function, which is, in our case, **tanh**. So let's define it.

```

// tanh activation function
double tanh_activation(double x){
    return (exp(2*x) - 1) / (exp(2*x) + 1);
}

```

We used the exponential form as it makes more sense and easy to implement in C. The last piece of code we need is for memory optimization. We need to free our **Neuron** after it returns its output.

```

void free_neuron(Neuron *n){
    free(n->w); // Free the allocated memory for weights
}

```

And that's all! We have implemented a single neuron. Now it's time to test it.

```
int main() {
```

```

    srand(time(NULL)); // Seed the random number generator

    Neuron n;

    double input[] = {1.0, 2.0, 3.0};

    init_neuron(&n, 3); // Initialize the neuron with 3 input dimensions
    double output = tanh_activation(call_neuron(&n, input)); // Calculate
    the output of the neuron and apply tanh activation

    printf("Output of a single neuron: %f\n", output);

    free_neuron(&n); // Free the allocated memory for the neuron
}

```

If we run this, we will get an output like this:

```

Output of a single neuron: 0.931407

```

And each time we run, the output will be different, as each time the weights are initialized randomly.

The Layer Structure:

Now that we have a single, functional neuron, the next step in our architecture is to group them. In a neural network, a **Layer** is simply a collection of neurons that all operate in parallel on the same input data.

For example, if our layer has 3 inputs and 2 output neurons, both of those neurons will receive the same 3 input values. They will each multiply those inputs by their own unique, independently initialized weights, resulting in 2 distinct output values.

To represent this in C, we define a *Layers* structure. Instead of hardcoding a fixed number of neurons, we use a pointer to dynamically allocate an array of neurons based on the desired architecture.

```

typedef struct {
    Neuron *neurons;
    int num_inputs;
    int num_outputs;
}Layers;

```

When we initialize a layer, we must allocate memory for the array of neurons. Then, we iterate through that array and initialize each individual neuron, telling it how many inputs to expect so it can allocate its own weight array.

```

void init_layer(Layers *l, int num_inputs, int num_outputs){
    l->num_inputs = num_inputs;
    l->num_outputs = num_outputs;
    l->neurons = (Neuron*) malloc(num_outputs * sizeof(Neuron));

    for(int i = 0; i < num_outputs; i++){
        init_neuron(&l->neurons[i], num_inputs); // Initialize each neuron
in the layer
    }
}

```

The forward pass for a layer involves taking an array of input values and feeding it to every single neuron. We must collect all of their individual activated outputs into a new array, which will then serve as the input for the *next* layer in the network.

```

double* call_layer(Layers *l, double *in){
    double *out = (double*) malloc(l->num_outputs * sizeof(double));
    for(int i = 0; i < l->num_outputs; i++){
        out[i] = tanh_activation(call_neuron(&l->neurons[i], in)); //
Calculate the output of each neuron and apply tanh activation
    }

    return out; // Return the output of the layer
}

```

Now, we use a function to free the memory

```

void free_layer(Layers *l){
    for(int i = 0; i < l->num_outputs; i++){
        free_neuron(&l->neurons[i]); // Free each neuron in the layer
    }
    free(l->neurons); // Free the allocated memory for neurons
}

```

Now, for experimental purposes, let's write an execution code and see how it works.

```

int main() {
    srand(time(NULL)); // Seed the random number generator

    Layers layer;
    init_layer(&layer, 3, 2); // Initialize a layer with 3 inputs and 2
outputs
    double input[3] = {0.5, -0.2, 0.1}; // Example input
    double *output = call_layer(&layer, input); // Get the output from the
layer
    printf("Output of the layer:\n");
}

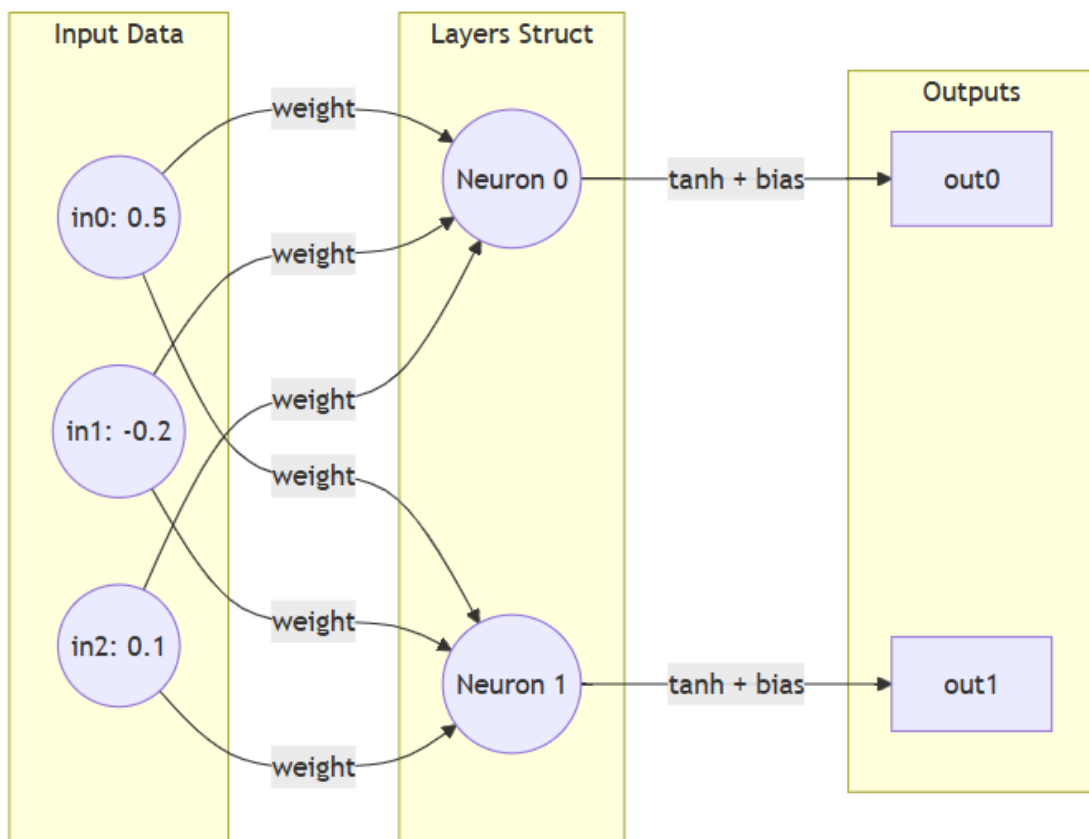
```

```

for(int i = 0; i < layer.num_outputs; i++){
    printf("%f\n", output[i]); // Print the output of each neuron in the
    layer
}
free(output); // Free the allocated memory for the output
}

```

Here, we used a layer with 3 inputs and 2 outputs. If we visualize it, this is what we created:



And the output will be something like:

```

Output of the layer:
-0.695719
0.811923

```

Until now, we have created a layer full of neurons. Now we need our full **MLP** (Multi-Layer Perceptron), or we can say array or layers, and this is what we create in libraries like **TensorFlow** or **PyTorch**. If we recall, we write,

```

import tensorflow as tf

```



```

from tensorflow.keras import layers, models

# Define a simple multi-layer neural network
model = models.Sequential([
    # Input layer accepts 20 features; maps to 64 hidden units with ReLU
    layers.Dense(64, activation='relu', input_shape=(20,)),

    # Hidden layer with 32 units
    layers.Dense(32, activation='relu'),

    # Output layer with 10 units for multi-class classification
    layers.Dense(10, activation='softmax')
])

# View the structural dimensions and parameter count
model.summary()

```

The 3 lines, inside Sequential, are nothing but a stack of layers. This is what we want to create now.

NOTE: The layers we have written are actually Dense Layers. To implement other layers, such as Dropout or Conv2D, we need to write separate logic

The Multi-Layer Perceptron (MLP):

With our neurons and layers functioning, we have reached the final architectural stage: connecting multiple layers together to form a deep neural network, formally known as a Multi-Layer Perceptron (MLP).

When we talk about network architecture, we typically describe it by the sizes of its layers. For example, an architecture of {3, 4, 2} means 3 inputs, a hidden layer of 4 neurons, and an output layer of 2 neurons. Even though there are 3 "sizes" here, there are only 2 physical layers of weights doing the math: the connections from the input to the hidden layer, and from the hidden layer to the output.

To represent the full network, we define an **MLP** struct that holds an array of our previously defined **Layers**.

```

typedef struct{
    Layers *layers;
    int num_layers;
}MLP;

```

num_layers will be the number of layers. We have to keep that in mind; all layers here are **Dense** layers.

```

void init_mlp(MLP *mlp, int num_layers, int *layer_sizes){
    mlp->num_layers = num_layers - 1; // 3 sizes means 2 layers of weights
    and one input layer
    mlp->layers = (Layers*) malloc(mlp->num_layers * sizeof(Layers));

    for(int i = 0; i < mlp->num_layers; i++){
        init_layer(&mlp->layers[i], layer_sizes[i], layer_sizes[i+1]); //
        Initialize each layer in the MLP
    }
}

```

When we initialize the network, we pass in the *total_layers* and an array of *layer_sizes*. The function calculates how many computational layers actually exist and allocates memory for them.

A forward pass through an entire network is like a relay race. The input data is passed to the first layer, which calculates an output array. That output array is immediately handed off to the next layer to be used as its input, and so on.

In C, we must be incredibly careful with memory here. Since every layer dynamically allocates a new array for its output, we must free the intermediate arrays as soon as the next layer is finished reading them.

```

double *call_mlp(MLP *mlp, double *in){
    double *current_input = in;
    double *current_output;

    for(int i = 0; i < mlp->num_layers; i++){
        current_output = call_layer(&mlp->layers[i], current_input); // Get
        the output of the current layer
        if(i > 0){
            free(current_input); // Free the previous layer's output if it's
            not the initial input
        }
        current_input = current_output; // The output of the current layer
        becomes the input for the next layer
    }

    return current_output; // Return the final output of the MLP
}

```

Notice the line *if(i > 0)*, this if block basically frees the previous layers after computation, which practically saves a lot of memory.

When the network has finished training or predicting, we must systematically tear down the architecture. We free the weights inside every neuron, then the neurons inside every layer, and finally the layers inside the MLP.

```
void free_mlp(MLP *mlp){
    for(int i = 0; i < mlp->num_layers; i++){
        free_layer(&mlp->layers[i]); // Free each layer in the MLP
    }
    free(mlp->layers); // Free the allocated memory for layers
}
```

Finally, we're done with our basic MLP. It can now successfully process data. But what we have implemented till now is the **forward propagation**.

Backpropagation:

Our network can now make predictions, but those predictions are currently complete garbage because our weights are completely random. To make the network learn, we need a mechanism to measure exactly how wrong it is, figure out which specific weights caused the error, and adjust them.

This process requires four consecutive steps: **Loss Calculation**, **Activation Derivatives**, **Backpropagation (The Chain Rule)**, and **Gradient Descent**.

Loss Calculation:

To know how much to change our weights, we first need a single number that represents our total error. For this implementation, we need a mathematical function. While **Cross-Entropy** is standard for classification (like **MNIST**), **MSE** is mathematically simpler to implement from scratch and perfectly demonstrates the core mechanics of gradient flow.

Mean Squared Error (MSE):

In [statistics](#), the **mean squared error (MSE)** or **mean squared deviation (MSD)** of an [estimator](#) (of a procedure for estimating an unobserved quantity) measures the [average](#) of the squares of the [errors](#).

The formula for MSE across an output layer is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

(Where n is the number of predictions generated by a sample of n data points, $\mathbf{Y}$ is the vector of observed values of the variable being predicted, and $\hat{\mathbf{Y}}$ is the predicted values.)

To start backpropagation, we need the *derivative* of this loss with respect to our prediction. Using basic calculus, the derivative simplifies to:

$$\frac{\partial Loss}{\partial Prediction} = 2 \times (Prediction - Target)$$

(Note: Many implementations drop the **2** or the **1/n** during the backward pass because the Learning Rate absorbs constant scalars anyway, but we will keep it for mathematical purity.)

The Derivative of Tanh:

Backpropagation flows backward through our activation functions. This means we need the derivative of Tanh.

One of the most beautiful properties of Tanh is that its derivative can be calculated using its own output. If $a = \tanh(z)$, then the derivative is:

$$\frac{\partial \tanh(z)}{\partial z} = 1 - a^2$$

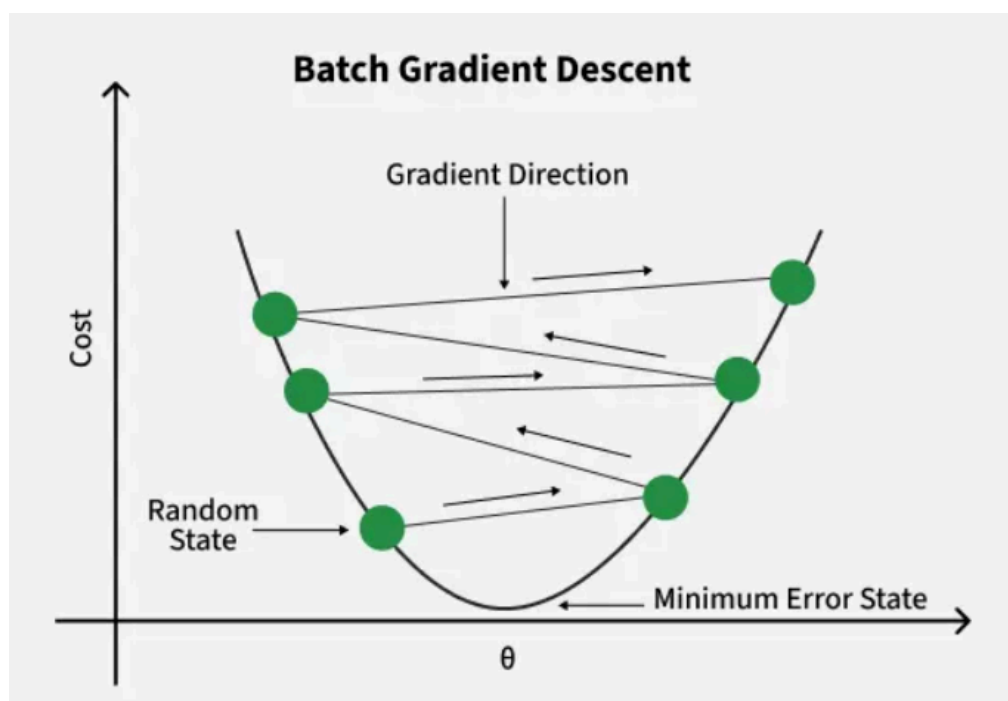
Because our forward pass already calculated and stored the activated output, finding the derivative during the backward pass is extremely computationally cheap. We don't have to recalculate the exponentials; we just square the cached output and subtract it from 1.

Gradient Descent:

Once backpropagation successfully calculates ∂w the ∂b variables for every neuron, the final step is incredibly simple. We loop through the network and nudge the weights in the opposite direction of the gradient, scaled by a small **learning rate**.

$$W_{new} = W_{old} - (\text{Learning rate} \times \partial W)$$

This is how we adjust our weights and biases for every neuron to predict the actual output.



Upgrading the Architecture for Training:

Before we can write the backpropagation loops, we must upgrade our C structures. During the backward pass, every single weight and bias needs to accumulate its own specific gradient.

Because C doesn't have an autograd engine to track these invisibly, we must manually add memory allocation for gradients directly into our **Neuron**.

```
// Definition of our Neuron structure
typedef struct {
    int input_dim;
    double *w;
    double b;

    double *dw; // Gradient of weights
    double db; // Gradient of bias

    double output; // The activated output (a) needed for the Tanh
    derivative
}Neuron;
```

With this architectural change, our **init_neuron** function must now also allocate space for **dw** and **db** and initialize it with 0.

```
// Initializes the neuron with random weights and bias
void init_neuron(Neuron *n, int input_dim){
    n->input_dim = input_dim;
    n->w = (double*) malloc(input_dim * sizeof(double));
    n->dw = (double*) malloc(input_dim * sizeof(double)); // Allocate memory
    for weight gradients
    for(int i = 0; i < input_dim; i++){
        n->w[i] = ((double)rand() / RAND_MAX) * 2.0 - 1.0; // Random weights
        between -1 and 1
        n->dw[i] = 0.0; // Initialize weight gradients to 0
    }
    n->b = ((double)rand() / RAND_MAX) * 2.0 - 1.0; // Random bias between
    -1 and 1
    n->db = 0.0; // Initialize bias gradient to 0
    n->output = 0.0; // Initialize output to 0
}
```

Crucially, we must also update `free_neurons` to `free(n->dw)` to avoid memory leaks

Now we need to store the output of each neuron in the **call_layer** function,

```
double* call_layer(Layers *l, double *in){
    double *out = (double*) malloc(l->num_outputs * sizeof(double));
    for(int i = 0; i < l->num_outputs; i++){
        l->neurons[i].output = call_neuron(&l->neurons[i], in); // Calculate
the output of each neuron
        out[i] = tanh_activation(l->neurons[i].output); // Apply tanh
activation
    }

    return out; // Return the output of the layer
}
```

Now, to calculate the gradient of a specific weight, we desperately need to know what the input to that layer was. But in our current **call_mlp** function, we free as soon as the layer finishes! We need to cache those inputs. To do this, we need to add an **input catch** in our **Layers** structure.

```
typedef struct {
    Neuron *neurons;
    int num_inputs;
    int num_outputs;

    double *inputs_cache; // Cache the inputs for backpropagation
}Layers;
```

Now, while initialization, we need to set the catch as NULL. and while freeing the memory we also need to free the catch.

```
void init_layer(Layers *l, int num_inputs, int num_outputs){
    l->num_inputs = num_inputs;
    l->num_outputs = num_outputs;
    l->inputs_cache = NULL; // Initialize the input cache to NULL
    l->neurons = (Neuron*) malloc(num_outputs * sizeof(Neuron));

    for(int i = 0; i < num_outputs; i++){
        init_neuron(&l->neurons[i], num_inputs); // Initialize each neuron
in the layer
    }
}
```

```
void free_layer(Layers *l){
    for(int i = 0; i < l->num_outputs; i++){
        free_neuron(&l->neurons[i]); // Free each neuron in the layer
    }
}
```

```

    free(l->neurons); // Free the allocated memory for neurons
    if (l->inputs_cache != NULL) free(l->inputs_cache); // Free the input
    cache if it was allocated
}

```

Now let's store the catch while calling the layer,

```

double *call_layer(Layers *l, double *in) {
    if (l->inputs_cache != NULL) free(l->inputs_cache);

    l->inputs_cache = (double *)malloc(l->num_inputs * sizeof(double));
    for (int i = 0; i < l->num_inputs; i++) {
        l->inputs_cache[i] = in[i];
    }

    double *out = (double *)malloc(l->num_outputs * sizeof(double));
    for (int i = 0; i < l->num_outputs; i++) {
        double raw_z = call_neuron(&l->neurons[i], in);
        l->neurons[i].output = tanh_activation(raw_z);
        out[i] = l->neurons[i].output;
    }

    return out;
}

```

Now we write the heavy matrix calculus. `backward_layer` calculates the error for its own weights and returns an array of gradients to pass backward to the previous layer.

```

double* backward_layer(Layers *l, double *d_outputs) {
    // Allocate memory for the gradients we will pass backwards
    double *d_inputs = (double*) malloc(l->num_inputs * sizeof(double));
    for(int i = 0; i < l->num_inputs; i++) d_inputs[i] = 0.0;

    for(int i = 0; i < l->num_outputs; i++) {
        Neuron *n = &l->neurons[i];

        // 1. The derivative of Tanh: (1 - a^2)
        double dtanh = 1.0 - (n->output * n->output);

        // 2. The total error for this specific neuron (Chain Rule)
        double error = d_outputs[i] * dtanh;

        // 3. Accumulate gradients for weights and biases
        for(int j = 0; j < n->input_dim; j++) {

```

```

        n->dw[j] += error * l->inputs_cache[j]; // Gradient for this
weight
        d_inputs[j] += error * n->w[j];          // Pass error
backward to previous neuron
    }
    n->db += error; // Gradient for bias is just the error
}

return d_inputs;
}

```

Just like `call_mlp` iterates forwards, `backward_mlp` iterates backwards from the output layer to the input layer, carefully freeing intermediate gradient arrays so we don't leak memory.

Once backpropagation finishes, every single neuron has its **dw** and **db** filled out. We just loop through and adjust the weights by subtracting a fraction (the learning rate) of the gradient. This is the job of optimizers.

```

void update_mlp(MLP *mlp, double learning_rate) {
    for(int i = 0; i < mlp->num_layers; i++) {
        Layers *l = &mlp->layers[i];
        for(int j = 0; j < l->num_outputs; j++) {
            Neuron *n = &l->neurons[j];

            // Adjust weights and reset gradients for the next training
Loop
            for(int k = 0; k < n->input_dim; k++) {
                n->w[k] -= learning_rate * n->dw[k];
                n->dw[k] = 0.0;
            }

            // Adjust bias and reset gradient
            n->b -= learning_rate * n->db;
            n->db = 0.0;
        }
    }
}

```

What we just wrote is standard **Vanilla Gradient Descent** (or SGD - Stochastic Gradient Descent, depending on your batch size). It only looks at the **current** gradient. If the gradient is steep, it takes a big step. If the gradient is shallow, it takes a small step. The optimizers like **Adam** are much more complex to build.

Finally, we build our **loss function**,


```
double MSE(double *predictions, double *targets, int size){
    double sum = 0;
    for(int i = 0; i < size; i++){
        sum += pow(predictions[i] - targets[i], 2); // Calculate the squared difference
    }
    return sum / size; // Return the mean of the squared differences
}
```

Congratulations, we have built our own neural network from scratch.

Now, it's time to solve a basic problem with this:

Using our own Neural Net:

To test this, let's solve a classification problem. We will define a random dataset then train our model on it.

```
int main() {
    srand(time(NULL));
    double xs[4][3] = {
        {2.0, 3.0, -1.0},
        {3.0, -1.0, 0.5},
        {0.5, 1.0, 1.0},
        {1.0, 1.0, -1.0}
    }; // 4 samples, each with 3 features

    double ys[4] = {1.0, -1.0, -1.0, 1.0}; // 4 target values
}
```

Now we add layer sizes and implement our MLP.

```
int layer_sizes[3] = {3, 4, 1};
MLP mlp;
init_mlp(&mlp, 3, layer_sizes);
```

For this, let's do a 100 iteration.

```
int epochs = 1000;
double learning_rate = 0.01;
for(int epoch = 0; epoch < epochs; epoch++) { }
```

Now, inside the for loop, we predict each target value, then calculate the loss and use gradient descent to adjust weights and biases.

```
for(int epoch = 0; epoch < epochs; epoch++) {
```

```

double total_loss = 0.0;

for (int i = 0; i < 4; i++){
    double *pred = call_mlp(&mlp, xs[i]);
    double loss = MSE(pred, &ys[i], 1);
    total_loss += loss;

    double *d_loss = (double*) malloc(1 * sizeof(double));
    d_loss[0] = 2.0 * (pred[0] - ys[i]);

    backward_mlp(&mlp, d_loss);
    update_mlp(&mlp, learning_rate);

    free(pred);
    free(d_loss);
}

if (epoch % 100 == 0) {
    printf("Epoch %d | Loss: %f\n", epoch, total_loss / 4.0);
}
}

```

Now, a final prediction check:

```

printf("\n--- Training Complete ---\n");
for (int i = 0; i < 4; i++) {
    double *pred = call_mlp(&mlp, xs[i]);
    printf("Input %d: Expected %f, Got %f\n", i, ys[i], pred[0]);
    free(pred);
}

free_mlp(&mlp);
return 0;

```

So, in my case, the output was:

```

Epoch 0 | Loss: 0.825672
Epoch 100 | Loss: 0.348607
Epoch 200 | Loss: 0.194229
Epoch 300 | Loss: 0.157960
Epoch 400 | Loss: 0.136801
Epoch 500 | Loss: 0.130599
Epoch 600 | Loss: 0.192583
Epoch 700 | Loss: 0.224903
Epoch 800 | Loss: 0.112650
Epoch 900 | Loss: 0.073064

--- Training Complete ---

```

```
Input 0: Expected 1.000000, Got 0.728275
Input 1: Expected -1.000000, Got -0.771050
Input 2: Expected -1.000000, Got -0.722923
Input 3: Expected 1.000000, Got 0.738839
```

As you can see, the losses are continuously decreasing as we wanted. An incredibly sharp observation is that we are solving a **binary classification** problem, but you are using **regression mechanics** to do it.

Solving the MNIST Digit Classification problem

Up until now, we have been using a tiny, hardcoded 4-sample dataset. To train a network to recognize handwriting, we need thousands of examples. The MNIST dataset provides exactly this: 60,000 training images and 10,000 testing images of handwritten digits (0-9).

Each image is a perfectly cropped, grayscale box measuring 28x28 pixels. Because 28 multiplied by 28 is 784, the input layer of our neural network will need exactly **784 input neurons**.

The Binary Format:

Unlike modern datasets that come in .csv files or .png folders, the original MNIST dataset is distributed in a raw binary format (often called .idx format).

Reading raw bytes in C using **fread()** is very fast, but MNIST introduces a classic low-level computer science trap: **Endianness**.

- Modern Intel, AMD, and Apple Silicon processors read binary data in **Little-Endian** format (least significant byte first).
- The MNIST files were written in **Big-Endian** format (most significant byte first) because they were created on older IBM/Motorola processors in the 1990s.

If we just read a 32-bit integer directly from the MNIST file, the number 60,000 will look like 16121856 to our computer because the bytes are backward! Before we parse the dataset, we must write a small C function to flip the bytes of every integer we read.

Here is a fast bitwise operation to reverse a 32-bit integer. It isolates each of the 4 bytes using a bitmask (& 255), shifts them to their opposite positions, and adds them back together.

```
// Flips a 32-bit integer from Big-Endian to Little-Endian
int reverse_int(int i) {
    unsigned char c1, c2, c3, c4;
    c1 = i & 255;
    c2 = (i >> 8) & 255;
    c3 = (i >> 16) & 255;
```

```

    c4 = (i >> 24) & 255;

    return ((int)c1 << 24) + ((int)c2 << 16) + ((int)c3 << 8) + c4;
}

```

Since we are doing Supervised Learning, every image needs to be permanently tethered to its true label (e.g., this array of pixels represents a '7'). We will create a struct to hold a single dataset sample.

```

typedef struct {
    double pixels[784]; // 28 * 28 pixels
    int label;           // The true digit (0-9)
} MNIST_Image;

```

The dataset is split across two files: one for the images and one for the labels. We will write a function that opens both files, reads the file headers (magic numbers and sizes), and then loops through the data byte by byte.

Because our Neural Network uses the Tanh activation function, it expects inputs in a small range. Raw pixel values go from 0 to 255. As we read them, we will divide by 255 to normalize them between 0.0 and 1.0.

```

MNIST_Image* load_mnist(const char *image_filename, const char
*label_filename, int *num_images) {
    FILE *image_file = fopen(image_filename, "rb");
    FILE *label_file = fopen(label_filename, "rb");

    if (!image_file || !label_file) {
        printf("Error: Could not open MNIST files.\n");
        exit(1);
    }

    // Read the headers (32-bit integers)
    int magic_number, num_items, num_rows, num_cols, label_magic,
label_items;

    fread(&magic_number, sizeof(int), 1, image_file);
    magic_number = reverse_int(magic_number);

    fread(&num_items, sizeof(int), 1, image_file);
    num_items = reverse_int(num_items);

    fread(&num_rows, sizeof(int), 1, image_file);
    num_rows = reverse_int(num_rows);

```

```

fread(&num_cols, sizeof(int), 1, image_file);
num_cols = reverse_int(num_cols);

// Read Label headers
fread(&label_magic, sizeof(int), 1, label_file);
fread(&label_items, sizeof(int), 1, label_file);
label_items = reverse_int(label_items);

// Allocate memory for the entire dataset
MNIST_Image *dataset = (MNIST_Image*) malloc(num_items *
sizeof(MNIST_Image));
*num_images = num_items;

// Read the data!
for (int i = 0; i < num_items; i++) {
    // 1. Read the Label (1 byte)
    unsigned char label;
    fread(&label, sizeof(unsigned char), 1, label_file);
    dataset[i].label = (int)label;

    // 2. Read the pixels (784 bytes)
    for (int p = 0; p < 784; p++) {
        unsigned char pixel;
        fread(&pixel, sizeof(unsigned char), 1, image_file);

        // Normalize from [0, 255] to [0.0, 1.0]
        dataset[i].pixels[p] = (double)pixel / 255.0;
    }
}

fclose(image_file);
fclose(label_file);
return dataset;
}

```

We have our Neural Network engine, and we have our MNIST data loader. For this task, our architecture will look significantly larger than our 4-sample test. We will build a network with **784 inputs** (one for each pixel), a **hidden layer of 128 neurons**, and an **output layer of 10 neurons** (one for each possible digit, 0-9).

Before we write the training loop, we need two small helper functions.

First, our dataset gives us integer labels (like **7**), but our network has 10 output neurons using Tanh. We must translate the integer **7** into a target array where the 7th index is **1.0**, and all other indices are **-0.1**.

```
// Translates a Label (0-9) into an array of targets for Tanh
void one_hot_encode(int label, double *target_array) {
    for (int i = 0; i < 10; i++) {
        target_array[i] = -1.0;
    }
    target_array[label] = 1.0; // Only the correct digit gets a 1.0
}
```

Second, when the network makes a prediction, it outputs an array of 10 floating-point numbers. To figure out which digit the network is actually guessing, we just find the index of the highest number. This is called **Argmax**.

```
// Returns the index (0-9) of the highest value in the output array
int argmax(double *array, int size) {
    int best_index = 0;
    double max_val = array[0];
    for (int i = 1; i < size; i++) {
        if (array[i] > max_val) {
            max_val = array[i];
            best_index = i;
        }
    }
    return best_index;
}
```

Finally, here is the master function. It loads the binary files, initializes the architecture, and runs a Stochastic Gradient Descent (SGD) training loop over the images.

(Note: Because we are writing in pure C without GPU acceleration, iterating through 60,000 images one by one is computationally heavy. For this example, we will train on a subset of the first 5,000 images so you can watch the accuracy improve in real-time.)

```
int main() {
    srand(time(NULL));

    // 1. Load the Data
    printf("Loading MNIST Dataset...\n");
    int num_images;
    // (Ensure you have downloaded train-images.idx3-ubyte and
    train-labels.idx1-ubyte)
    MNIST_Image *dataset = load_mnist("train-images.idx3-ubyte",
    "train-labels.idx1-ubyte", &num_images);
    printf("Successfully loaded %d images.\n\n", num_images);
}
```

```

    // 2. Build the Architecture (784 Pixels -> 128 Hidden -> 10
Outputs)
    int layer_sizes[] = {784, 128, 10};
    MLP mlp;
    init_mlp(&mlp, 3, layer_sizes);

    // 3. Hyperparameters
    int epochs = 10;
    int train_size = 5000; // Train on a subset for speed
    double learning_rate = 0.01;
    double target_array[10];

    // 4. The Training Loop
    printf("Beginning Training Loop...\n");
    for (int epoch = 0; epoch < epochs; epoch++) {
        int correct_predictions = 0;
        double total_loss = 0.0;

        for (int i = 0; i < train_size; i++) {
            // Encode the target
            one_hot_encode(dataset[i].label, target_array);

            // Forward Pass
            double *pred = call_mlp(&mlp, dataset[i].pixels);

            // Check Accuracy (Did the network guess right?)
            int guess = argmax(pred, 10);
            if (guess == dataset[i].label) {
                correct_predictions++;
            }

            // Calculate MSE Loss for all 10 output neurons
            total_loss += MSE(pred, target_array, 10);

            // Backward Pass
            double *d_loss = (double*) malloc(10 * sizeof(double));
            for(int j = 0; j < 10; j++){
                d_loss[j] = 2.0 * (pred[j] - target_array[j]);
            }

            backward_mlp(&mlp, d_loss);
            update_mlp(&mlp, learning_rate);

            free(pred);
            free(d_loss);
        }
    }

```

```

        // Print Epoch Statistics
        double accuracy = ((double)correct_predictions / train_size) *
100.0;
        printf("Epoch %d | Loss: %f | Accuracy: %.2f%%\n", epoch + 1,
total_loss / train_size, accuracy);
    }

    // 5. Memory Cleanup
    printf("\nCleaning up memory...\n");
    free_mlp(&mlp);
    free(dataset);

    return 0;
}

```

Make sure to download and unzip the **MNIST** dataset from Kaggle.

<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

The output of this program should look like this,

```

Loading MNIST Dataset...
Successfully loaded 60000 images.

Beginning Training Loop...
Epoch 1 | Loss: 0.334851 | Accuracy: 35.20%
Epoch 2 | Loss: 0.259940 | Accuracy: 45.48%
Epoch 3 | Loss: 0.245975 | Accuracy: 47.74%
Epoch 4 | Loss: 0.235176 | Accuracy: 49.98%
Epoch 5 | Loss: 0.225596 | Accuracy: 54.80%
Epoch 6 | Loss: 0.200880 | Accuracy: 58.82%
Epoch 7 | Loss: 0.193259 | Accuracy: 60.12%
Epoch 8 | Loss: 0.185790 | Accuracy: 60.72%
Epoch 9 | Loss: 0.180254 | Accuracy: 62.52%
Epoch 10 | Loss: 0.171243 | Accuracy: 64.44%

Cleaning up memory...

```

Of course, if you tweak the hyperparameters and use the whole data, accuracy would be increased. For simplicity, I just showed that model, and our architecture is functional.

Congratulations if you have made it this far. Now you know how Neural nets actually work under the hood, below level 0.

GitHub link to this project: [Neural Network in C](#)

Author: Aritra Das